

FREE

Arduino Starter Activity

Blinking LED + Student Worksheet

STEM Coding Introduction | Grades 5 – 8

Introduce students to physical computing with their first Arduino program.

A simple, engaging lesson that connects coding, circuits, and problem solving.

What is included:

Beginner-friendly coding lesson with step-by-step guidance
Hands-on electronics circuit building activity
Student worksheet, teacher guide, and answer key included

No prior experience required.

Suitable for first-time Arduino users. Compatible with all standard Arduino Uno boards.

The graphic features the title "ARDUINO STARTER ACTIVITY" in large, bold, teal letters, with a lightbulb icon to the right. Below the title is a dark blue banner with the text "Blinking LED + Student Worksheet" in white. Underneath the banner are four circular icons: a blue infinity symbol for "HANDS-ON STEM", a yellow lightbulb for "BEGINNER FRIENDLY", a green document with a pencil for "STUDENT WORKSHEET INCLUDED", and a purple laptop for "PERFECT FOR GRADES 5-8". Below these icons is a photograph of an Arduino Uno board with a red LED connected to a breadboard. In the bottom left corner, there is a green gift icon with the text "FREEBIE! PERFECT FOR INTRODUCING ARDUINO". To the right of the photo is a preview of the student worksheet, which includes sections for "PART 1: CIRCUIT DIAGRAM", "PART 2: CODE ANALYSIS", and "PART 3: THINK & REFLECT". At the bottom of the graphic is a dark blue bar with four white checkmarks and the text "EASY SETUP", "REAL CODING", "ENGAGING & FUN", and "NO EXPERIENCE NEEDED".

ARDUINO STARTER ACTIVITY

Blinking LED + Student Worksheet

HANDS-ON STEM

BEGINNER FRIENDLY

STUDENT WORKSHEET INCLUDED

PERFECT FOR GRADES 5-8

FREEBIE!
PERFECT FOR INTRODUCING ARDUINO

Arduino Starter Activity
Blinking LED

PART 1: CIRCUIT DIAGRAM
Draw the circuit used to make the LED blink.

PART 2: CODE ANALYSIS
1. What does the following line of code do?
`digitalWrite(13, HIGH);`
2. How long does the LED stay ON in this program?
3. How long does the LED stay OFF in this program?

PART 3: THINK & REFLECT
4. What would happen if you changed the pin number from 13 to 9? Why?

EASY SETUP | REAL CODING | ENGAGING & FUN | NO EXPERIENCE NEEDED

Teacher Overview

Lesson Summary

In this introductory activity, students will build a simple electronic circuit and write their first Arduino program. The program instructs a microcontroller to turn an LED on and off repeatedly, creating a blinking light. Through this hands-on experience, students are introduced to foundational concepts in physical computing, including basic circuit construction, programming syntax, and the relationship between code and electronic output.

Learning Objectives

By the end of this activity, students will be able to:

1. Explain what an Arduino microcontroller is and what it does.
2. Identify the components in a basic LED circuit.
3. Write and upload a simple Arduino program to a microcontroller.
4. Describe the purpose of the `setup()` and `loop()` functions.
5. Observe and record the behavior of an output device (LED).
6. Troubleshoot basic circuit and code issues.

Estimated Time

Duration

30 to 45 minutes (can be extended across two class periods for younger students)

Materials Needed

Each student or group will need the following:

Hardware	Software
Arduino Uno (or compatible board)	Arduino IDE (free download: arduino.cc)
USB A-to-B cable	
Half-size or full-size breadboard	
1 x LED (any color)	
1 x 220-ohm or 330-ohm resistor	
Jumper wires (minimum 2)	

Teacher Tips

1. Pre-install the Arduino IDE on classroom computers before the lesson to save time.
2. Test one complete circuit yourself before class so you can troubleshoot student issues quickly.
3. Group students in pairs so they can support each other during circuit building.
4. Remind students that LEDs are directional. The longer leg (anode) always connects toward positive voltage.
5. If a student's LED does not blink, check the LED orientation first, then verify the connection to Pin 13.

Arduino Basics

Student-Friendly Introduction

What Is an Arduino?

An Arduino is a small, inexpensive electronic board that contains a microcontroller. It is designed to make it easy for beginners to learn electronics and programming. Arduino boards are used around the world in schools, makerspaces, and professional engineering environments.

Think of an Arduino as a tiny, programmable computer that connects directly to electronic components such as lights, motors, sensors, and buttons.

What Is a Microcontroller?

A microcontroller is a small computer on a single chip. Unlike a desktop computer, a microcontroller is built to perform one specific task repeatedly. It reads input (such as a button press or a sensor reading) and produces output (such as turning on a light or moving a motor).

The Arduino Uno uses the ATmega328P microcontroller. When you write a program and upload it to your Arduino, the microcontroller stores and runs that program continuously.

How Does Code Control Electronics?

When you write code on your computer and upload it to the Arduino, you are giving the microcontroller a set of instructions. The microcontroller follows these instructions in order and repeats them as long as the board has power.

In this activity, your code will tell the Arduino to send electricity to Pin 13, which is connected to an LED. When electricity flows through the LED, it lights up. When electricity stops, it turns off. By controlling when electricity flows and for how long, you can make the LED blink.

Arduino	Program (Code)	Result
Microcontroller board	Instructions you write	LED blinks on and off

STEM Concept: Output Devices

An LED (Light Emitting Diode) is an output device. Output devices receive signals from a controller and convert them into something observable. In this case, the Arduino sends an electrical signal to the LED, which converts that signal into visible light. Understanding output devices is a foundational concept in both electronics and computer science.

Circuit Instructions

Why Do You Need a Resistor?

Important Concept: An LED can only handle a small amount of electrical current before it burns out. A resistor limits the amount of current flowing through the LED, protecting it from damage. Always place a resistor in series with an LED. For this activity, use a 220-ohm or 330-ohm resistor.

Teacher Tip: Troubleshooting LED Orientation

The most common beginner mistake is inserting the LED backwards. An LED is a directional component — it will only light up when connected the correct way. When troubleshooting a circuit that does not work, always check the LED orientation first. The longer leg (anode) must connect toward the positive voltage side of the circuit (toward the resistor and Pin 13). The shorter leg (cathode) connects to GND. Flipping the LED around is a quick and easy first fix.

Step-by-Step Circuit Assembly

1. Connect your Arduino to your computer using the USB cable. Do not open the Arduino IDE yet.
2. Place the LED into the breadboard. The longer leg of the LED is the positive leg (anode). The shorter leg is the negative leg (cathode).
3. Insert one end of the resistor into the same breadboard row as the longer leg (positive/anode) of the LED.
4. Connect a jumper wire from the other end of the resistor to Pin 13 on the Arduino.
5. Connect a second jumper wire from the shorter leg (negative/cathode) of the LED to any GND (ground) pin on the Arduino.
6. Double-check all connections before proceeding to the next section.

Component	Connects To
LED — long leg (anode)	Resistor (in series)
LED — short leg (cathode)	GND pin on Arduino
Resistor (opposite end)	Pin 13 on Arduino
USB cable	Computer (for power and programming)

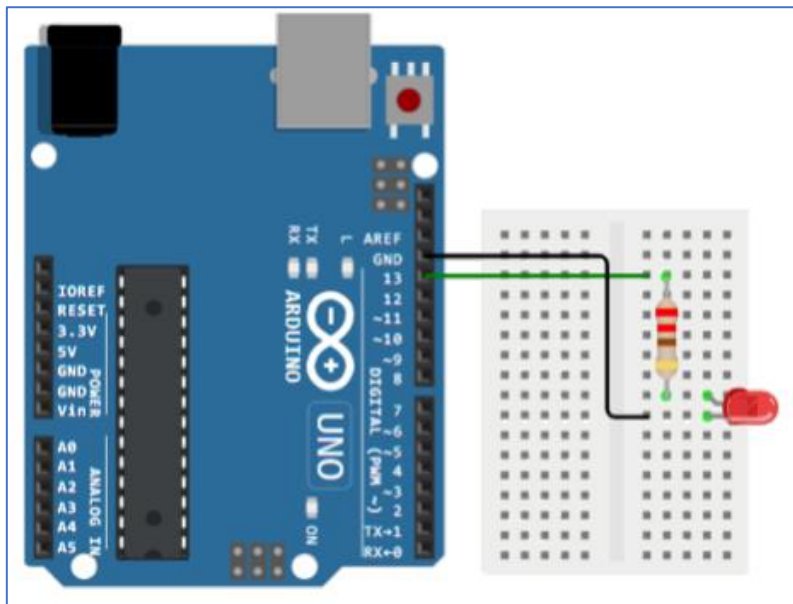


Figure 1: Arduino Blinking LED Circuit — Breadboard Layout

The Arduino Code

The Blinking LED Program

Open the Arduino IDE on your computer. Type or copy the following code into the editor window, then click the Upload button (right-arrow icon) to send the program to your Arduino board.

```
// Blinking LED — Starter Program

void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

Line-by-Line Code Explanation

Code	What It Does
<code>void setup() { }</code>	This function runs once when the Arduino is powered on or reset. Use it to configure settings before the main program begins.
<code>pinMode(13, OUTPUT);</code>	Tells the Arduino that Pin 13 will be used as an output. Electricity can be sent out of this pin to power the LED.
<code>void loop() { }</code>	This function runs repeatedly — over and over — as long as the Arduino has power. All main program logic goes here.
<code>digitalWrite(13, HIGH);</code>	Sends 5 volts to Pin 13, allowing current to flow through the circuit and turning the LED on.
<code>delay(1000);</code>	Pauses the program for 1000 milliseconds (1 second). Nothing else happens during this time.
<code>digitalWrite(13, LOW);</code>	Sets Pin 13 to 0 volts, stopping the flow of current and turning the LED off.

Student Worksheet

Name: _____ Date: _____ Period: _____

Part 1: Prediction

Prediction Question:

Before uploading the program, what do you think will happen when the code runs?
Describe what you expect to see.

I predict that:

Part 2: Observation Table

Upload the code and observe what happens. Record your observations for each trial in the table below.

Trial	LED Behavior	Notes / Questions
1	Original code (delay: 1000 ms)	
2	Changed delay to a smaller value	
3	Your own modification	

Part 3: Reflection Questions

Answer the following questions in complete sentences.

1. What does the `delay()` command do in the program? Why is it important?

2. What happened when you changed the delay value to a smaller number? What happened when you made it larger?

3. Why does the LED turn on and off repeatedly instead of staying on all the time?

4. What is the difference between the `setup()` function and the `loop()` function?

5. If you wanted the LED to blink faster, what specific change would you make to the code?

Challenge Extension Activities

Finished early? Try one or more of the challenges below. Each challenge requires you to modify the code you have already written. Record what you changed and what happened.

Challenge 1: Change the Blinking Speed

Task: Modify the `delay()` values in the code to make the LED blink faster, then slower.

Try a delay of 200 milliseconds. Then try 2000 milliseconds. Record what you observe in each case.

Tip: 1000 milliseconds = 1 second. A smaller number means a shorter pause and a faster blink.

Challenge 2: Create a Heartbeat Pattern

Task: Modify the code so the LED blinks in a heartbeat pattern: two quick blinks followed by a longer pause.

Hint: Add extra `digitalWrite()` and `delay()` lines inside the `loop()` function. Try the pattern below:

```
digitalWrite(13, HIGH); delay(150);  
digitalWrite(13, LOW); delay(100);  
digitalWrite(13, HIGH); delay(150);  
digitalWrite(13, LOW); delay(700);
```

Challenge 3: Blink Three Times, Then Pause

Task: Write a program that blinks the LED three times quickly, then waits two seconds before repeating.

Extension: Can you modify the code to blink exactly five times before the long pause?

Reflection: What would need to change in your code to increase the number of blinks without rewriting every line? (Hint: think about loops.)

Teacher Answer Guide

For teacher use only. Do not distribute to students.

Prediction Question

Acceptable answers include: the LED will turn on and then off, the light will blink, or the LED will flash on and off repeatedly. Accept any reasonable prediction that shows the student attempted to interpret the code or setup instructions.

Reflection Question Answers

Question 1: What does the `delay()` command do?

The `delay()` command pauses the program for a set amount of time. The value inside the parentheses is measured in milliseconds (1000 ms = 1 second). It is important because without it, the LED would turn on and off so rapidly that the human eye could not perceive the blinking.

Question 2: What happened when you changed the delay value?

A smaller delay value (such as 100) caused the LED to blink much faster. A larger value (such as 2000) made the LED blink more slowly. Students should connect the delay value directly to the speed of the blinking pattern.

Question 3: Why does the LED turn on and off repeatedly?

The `loop()` function runs continuously as long as the Arduino is powered. Each pass through the loop turns the LED on, waits, turns it off, and waits again. Because this cycle repeats indefinitely, the LED appears to blink.

Question 4: What is the difference between `setup()` and `loop()`?

The `setup()` function runs only once when the Arduino starts and is used to configure settings (such as declaring Pin 13 as an output). The `loop()` function runs continuously after `setup()` finishes. All ongoing program behavior — such as blinking — is placed inside `loop()`.

Question 5: How would you make the LED blink faster?

To make the LED blink faster, the student would decrease the `delay()` values. For example, changing `delay(1000)` to `delay(200)` would produce five blinks per second instead of one.

Common Student Errors and Fixes

Problem	Likely Cause / Fix
LED does not light up at all	Check LED orientation (long leg to positive). Verify Pin 13 connection. Confirm resistor is in series.
LED stays on but does not blink	Check code was uploaded successfully. Confirm both <code>delay()</code> calls and both <code>digitalWrite()</code> calls are present.
Upload error in IDE	Verify the correct board and port are selected under Tools in the Arduino IDE menu.
Code does not compile	Look for missing semicolons, mismatched brackets, or incorrect capitalization of function names (Arduino is case-sensitive).

Explore More Arduino STEM Activities

Ready to take your students further?

These resources expand the skills introduced in this free starter activity and provide complete STEM project units for the classroom.

Beginner Arduino Project Pack

Recommended next step

Includes:

1. Traffic light LED sequence project
2. Push-button input activity
3. Buzzer melody programming
4. Temperature sensor display
5. Full teacher guides and student worksheets

Arduino Intermediate Project Pack

For students ready to level up

Includes:

1. LCD display programming project
2. Servo motor control activity
3. Ultrasonic distance sensor project
4. Multi-component engineering builds
5. Structured engineering design challenges

Search for "Arduino STEM Project Pack" on Teachers Pay Teachers to find the full series.

Thank you for downloading this free resource.

If this activity worked well in your classroom, please leave a review on TPT. Your feedback helps other educators discover quality STEM resources.

Continue Your Arduino STEM Journey

If your students enjoyed this starter activity, explore the full Arduino STEM curriculum series designed for middle school classrooms. These resources build progressively from beginner projects to more advanced engineering challenges using Arduino.

Arduino Beginner Project Pack | 4 STEM Projects for Grades 4–6

A beginner-friendly introduction to Arduino programming and basic electronics. Students learn to build circuits, control LEDs and sensors, and understand how code interacts with hardware.

Includes:

- 4 guided Arduino STEM projects
- Step-by-step wiring instructions
- Student worksheets
- Teacher guides and explanations
- Perfect for upper elementary and early middle school

Arduino STEM Unit | Beginner Arduino Projects | Middle School Engineering Curriculum

A structured project-based STEM unit designed to introduce middle school students to physical computing and engineering design. Students explore multiple Arduino projects while learning coding logic, circuit building, and problem solving.

Includes:

- Multiple scaffolded projects
- Engineering design challenges
- Student worksheets and reflection activities
- Teacher implementation guide

Arduino Intermediate Project Pack | 5 STEM Projects + Capstone | State Machines

Take students to the next level with intermediate Arduino systems and engineering challenges. Students develop deeper programming skills including structured logic and multi-component systems.

Includes:

- 5 intermediate Arduino projects
- State machine programming concepts
- Multi-sensor systems
- A capstone engineering design challenge
- Quiz and project rubric

My First Robot Brain (Freebie)

A fun introductory robotics resource where students explore how microcontrollers can act as the brain of a robot. Students learn the basic idea of inputs, outputs, and control logic that allow robots to interact with the world.

Perfect for introducing robotics concepts alongside Arduino.

- Introduces input/output concepts in a robotics context
- Suitable for early middle school and upper elementary students
- No prior robotics experience required

MATH-O-TRONICS

ABOUT THE STORE

Math-O-Tronics creates engaging STEM + Math integration resources designed for Grades 5–8 classrooms. Our mission is to make STEM learning hands-on, meaningful, and accessible through Arduino projects, engineering challenges, coding activities, data analysis labs, and real-world problem solving.

Every resource is designed to help teachers bring interactive learning into the classroom while supporting NGSS, CCSS, and technology integration standards. Many activities include printable worksheets, answer keys, differentiation support, and Tinkercad simulation alternatives for classrooms without physical hardware.

Whether you teach STEM, math, science, robotics, makerspace, or technology classes, Math-O-Tronics resources are built to help students learn by creating, experimenting, and thinking like real engineers and problem-solvers.

✓ Arduino Projects	✓ Engineering Challenges
✓ Coding Activities	✓ Printable Worksheets & Answer Keys

ABOUT THE AUTHOR

Ahmad Abdullah is a STEM educator and curriculum creator passionate about combining engineering, coding, electronics, and mathematics into engaging classroom experiences for middle school students.

With a focus on hands-on learning, Ahmad develops practical Arduino-based activities that help students connect abstract math and science concepts to real-world applications. His goal is to make STEM education interactive, accessible, and exciting for both teachers and students.

Through Math-O-Tronics, he creates classroom-tested resources that support creativity, critical thinking, collaboration, and problem-solving skills for the next generation of innovators.

“Helping students learn through creativity, engineering, and real-world STEM exploration.”

Follow this store on Teachers Pay Teachers to be notified when new Arduino STEM resources are released.